

TEST PLAN OUTLINE:

(IEEE 829 FORMAT)

Test Plan Identifier:.....	1
Introduction:.....	1
Test Items:.....	2
Features to be Tested:	2
Features NOT to be Tested:	2
Test Approach:.....	3
Item Pass/Fail Criteria:	3
Test Deliverables:.....	3
Testing Tasks:.....	4
Environmental Needs:.....	4
Responsibilities:	5
Schedule:.....	5
Risk Analysis:	5

Test Plan Identifier:

TP-BEMS-V1.0.0

Introduction:

In this document contains the outline of the test strategy, the scope of the testing, and how testing will be executed for the BankCore Enterprise Management System (BEMS), which is a multi-module console application written in C#. The BEMS was developed by BankCore Solutions (Pty) Ltd to manage core banking operations and calculations. During testing the current BEMS system will be tested in several ways, like functional and non-functional testing that will thoroughly test the different modules of the BankCore Enterprise Management System. The ultimate goal of this testing project is to execute comprehensive tests, by implementing different testing methods like MSTest testing, NUnit testing, and xUnit testing, on the BEMS application to ensure that the system is stable and secure, as well as to ensure that the system is mathematically correct and precise, and also to ensure that the system strictly follows business rules before it is released.

Test Items:

The following BEMS modules will be tested from BankCore.Core:

- AccountService.cs – which is the account management module
- AuthService.cs – which is the authentication module
- InterestCalculator.cs – which is the interest calculation module
- LoanService.cs – which is the Loan management module
- ReportingService.cs – which is the reporting generation module
- TransactionService.cs – which is the transaction management/engine module
- ValidationService.cs – which is the input validation module for the whole program.

Features to be Tested:

- Account management and operations like creating, updating, closing, and state transactions of a clients' account.
- Transactions, like making deposits, withdraws, transfers reversals and concurrent business rules enforcement
- Calculations, like testing the mathematical correctness and accuracy of the BEMS application when it calculates compound and simple interest, along with loan schedules and arrear payment penalties.
- System robustness, like testing how well the system handles exceptions, whether the validation rules work correctly, and testing to see how well the program functions under load.
- Security and user access, like testing the user authentication to ensure that only authenticated users can access the system, testing role-based access control to ensure that only the correct users have the correct access to data only meant for their role, as well as testing the password policy to see that it is enforced correctly.

Features NOT to be Tested:

The front end design/user interface of the BEMS program, as the focus of this testing phase is to test the accuracy of the program in terms of calculations, as well as testing whether the program correctly follows business rules.

The data store, and databases of the program will also not be tested as the program makes use of a coded, in-memory datastore to simulate the database of the program.

Furthermore, the hardware compatibility of the program will also not be tested, instead only the performance of the program will be tested.

The API and third-party integration will also not be tested during this testing phase.

Test Approach:

During this testing phase we will use a mixture of the following:

Black-box testing, where we test the code without reviewing the code and the modules of the program, during black-box testing we will make use of different testing methods like equivalence partitioning, decision tables, state transitions, and boundary value analysis.

White-box testing, where we will take a look at the code of the program and structure thereof, and then run tests based on what we know.

In the test approach, the following frameworks will be used:

- MSTest, for account management tests and authentication module tests
- NUnit, for transaction engine tests, and other engines like the report generation engine
- xUnit, for Interest calculator tests and loan processing tests.

Item Pass/Fail Criteria:

For a module to pass, the following criteria must be met:

- All of the test cases that were designed must be run must be executed and also has to pass
- It is also crucial to ensure that all modules are free of any major defects that could cause significant security risks.
- All modules should be able to run, and must execute at least one time.
- The program should be able to execute the tasks expected from it

Test Deliverables:

In this test phase the following will be delivered:

- Test Plan Document (in IEEE 829 Standard)
- Risk Register Spreadsheet
- Test Case Specifications/Designs
- Requirements Traceability Matrix
- Test cases delivered as C# test projects
- Test Execution Summary Table
- Defect Log

- Software Test Report
- Reflection Journal

Testing Tasks:

This testing will be broken up into 5 separate phases, which are as follows:

- Phase 1:
Environment Setup and Test Planning
Estimated Duration: 20 Hours
- Phase 2:
Test Case Design and Test Case Writing:
Estimated Duration: 35 Hours
- Phase 3:
Test Framework Implementation
Estimated Duration: 45 Hours
- Phase 4:
Test Execution and Defect Management
Estimated Duration: 30 Hours
- Phase 5:
Reporting and Portfolio Compilation
Estimated Duration: 20 Hours

Environmental Needs:

IDE: Visual Studio 2022 Community Edition

Framework: .NET 8 SDK

Version Control: Local Git Repository

Packages:

- NuGet:
 - MSTest
 - NUnit
 - xUnit
 - Moq
 - FluentAssertions
 - Coverlet.collector

Global Tool: dotnet-reportgenerator-globaltool

Responsibilities:

Design of Test Plan, Test Cases, and related documentation: Jaydee Venter – Junior Software Tester

Management of Environment: Jaydee Venter – Junior Software Tester

Test Execution Summary: Jaydee Venter – Junior Software Tester

Defect Log: Jaydee Venter – Junior Software Tester

Test Reporting: Jaydee Venter – Junior Software Tester

Schedule:

This testing phase will take place over a total of 150 hours, with several checkpoints to be completed along the way to ensure that all 5 phases stipulated in the 'Test Tasks' section of this document are completed.

Risk Analysis:

Refer to the RiskRegister.xlsx spreadsheet for a in detail table which contains identified risks in the BEMS application, the impact of the identified risks and ways which can be implemented to mitigate the identified risks.